ВЫЧИСЛИТЕЛЬНЫЕ МЕТОДЫ · ЛЕКЦИЯ 7

Введение в машинное обучение

Персептрон, нейронные сети и обучение

Лю Шисян

Кафедра теплофизики (Э6)
МГТУ им. Н.Э. Баумана
Москва / 2026

Место этой лекции в курсе



На занятии:

- задаём веса логических элементов и получаем XOR;
- вычисляем выход многослойной сети;
- обучаем линейную модель и сеть с помощью градиентов;
- для регрессии отделяем train, validation и test.

План

1. Персептрон
2. Нейронные сети
3. Многослойные нейронные сети
4. Обучение нейронных сетей
5. Практика и итоги

1

Персептрон

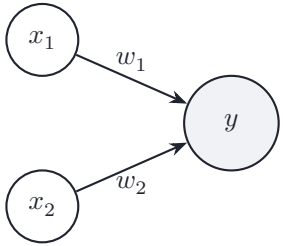
Персептрон

Определение

Персептрон — простейшая модель искусственного нейрона.
Он получает входные сигналы

$$x_1, x_2, \dots, x_d,$$

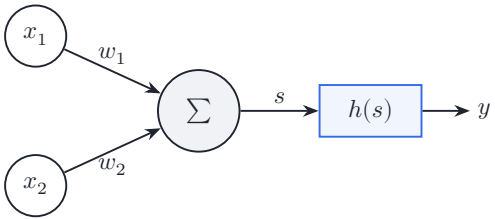
умножает их на веса и формирует один выход y.



- каждому входу x_i соответствует вес w_i;
- вес определяет **вклад входа** в решение;
- в классическом персептроне

$$y \in \{0, 1\}.$$

Математическая модель персептрона



1. Взвешенная сумма

$$s = w_1 x_1 + w_2 x_2.$$

2. Пороговое решение

$$y = \begin{cases} 0, & s \le \theta, \\ 1, & s > \theta. \end{cases}$$

- x_i — входные сигналы;
- w_i — веса;
- θ — порог активации.

Логический элемент AND (И)

Правило

AND выдаёт 1 **только тогда**, когда оба входа активны:

$$(x_1, x_2) = (1, 1) \quad \Longrightarrow \quad y = 1.$$

Выбираем параметры

$$(w_1, w_2, \theta) = (0.5,\ 0.5,\ 0.7),$$

поэтому

$$s = 0.5x_1 + 0.5x_2.$$

Только в последнем случае

$$s = 1.0 > \theta = 0.7.$$

Таблица истинности

x_1	x_2	y
0	0	0
1	0	0
0	1	0
1	1	**1**

Логический элемент NAND (НЕ–И)

Правило

NAND — отрицание AND:

$$(x_1, x_2) = (1, 1) \quad \Longrightarrow \quad y = 0,$$

а во всех остальных случаях $y = 1$.

Выбираем параметры

$$(w_1, w_2, \theta) = (-0.5,\ -0.5,\ -0.7),$$

поэтому

$$s = -0.5x_1 - 0.5x_2.$$

Только для двух активных входов

$$s = -1.0 \leq \theta = -0.7.$$

Таблица истинности

x_1	x_2	y
0	0	1
1	0	1
0	1	1
1	1	**0**

Логический элемент OR (ИЛИ)

Правило

OR выдаёт 1, если активен **хотя бы один** вход:

$$x_1 = 1 \quad \text{или} \quad x_2 = 1.$$

Ноль получается только при $(0, 0)$.

Выбираем параметры

$$(w_1, w_2, \theta) = (0.5,\ 0.5,\ 0.4),$$

поэтому

$$s = 0.5x_1 + 0.5x_2.$$

Уже один активный вход даёт

$$s = 0.5 > \theta = 0.4.$$

Таблица истинности

x_1	x_2	y
0	0	0
1	0	**1**
0	1	**1**
1	1	**1**

Порог и смещение

До сих пор решение записывалось через порог:

$$s = \mathbf{w}^T\mathbf{x}, \qquad y = \begin{cases} 0, & s \le \theta, \\ 1, & s > \theta. \end{cases}$$

Удобнее перенести порог внутрь линейного выражения:

$$z = \mathbf{w}^T\mathbf{x} + b, \qquad b = -\theta$$

Тогда

$$y = \begin{cases} 0, & z \le 0, \\ 1, & z > 0. \end{cases}$$

Веса w

Определяют, **как входные признаки влияют** на результат:

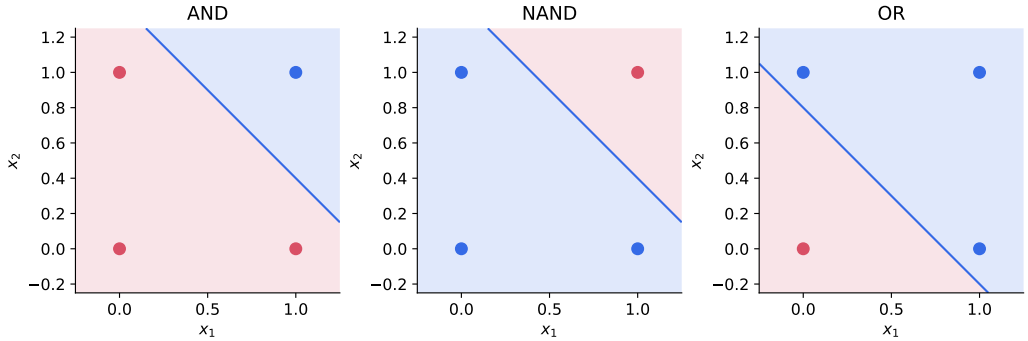
$$w_i x_i.$$

Смещение b

Сдвигает границу решения и определяет, **где происходит переключение** между классами.

Геометрический смысл персептрона

$w_1 x_1 + w_2 x_2 + b = 0$ — прямая, разделяющая два класса.



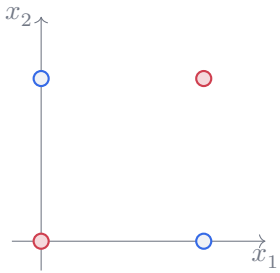
Синий: 1. Красный: 0. Фон показывает решение модели, точки — четыре входные комбинации.

Линейная неразделимость XOR

Исключающее ИЛИ

XOR выдаёт 1, когда **ровно один** из входов равен 1.

x_1	x_2	y
0	0	0
1	0	**1**
0	1	**1**
1	1	0



- точки двух классов расположены по диагонали;
- **ни одна прямая** не может отделить синие точки от красных;
- следовательно, XOR — **линейно неразделимая** задача.

Составление XOR из логических элементов

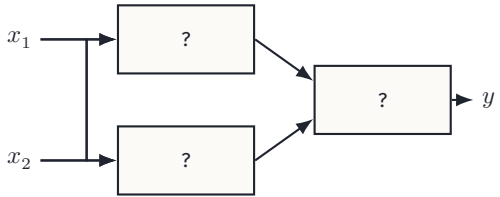
Мы уже умеем реализовывать с помощью персептрона три функции:

AND, NAND, OR.

Задача

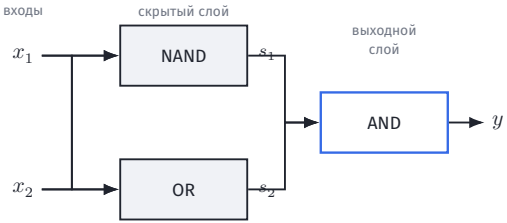
Можно ли **соединить несколько простых элементов** так, чтобы получить функцию XOR?

Выход должен быть равен 1, **только если один из двух входов равен** 1.



Подсказка: сначала сформируйте два промежуточных сигнала s_1 и s_2, затем объедините их.

XOR с помощью нескольких персептронов



$$
\begin{aligned}
s_1 &= \mathsf{NAND}(x_1, x_2),\\
s_2 &= \mathsf{OR}(x_1, x_2),\\
y &= \mathsf{AND}(s_1, s_2).
\end{aligned}
$$

Проверка

x_1	x_2	s_1	s_2	y
0	0	1	0	0
1	0	1	1	**1**
0	1	1	1	**1**
1	1	0	1	0

Теперь используются **несколько связанных персептронов**.
NAND и OR формируют промежуточные сигналы s_1 и s_2.
Они образуют **скрытый слой**, а AND формирует выход y.

Логические элементы в NumPy

```
X = np.array([[0, 0], [1, 0], [0, 1], [1,
     1]])

def step(z):
    return (np.asarray(z) > 0).astype(int)

def perceptron(X, weights, bias):
    return step(X @ np.asarray(weights) +
     bias)
```

```
h1 = perceptron(X, *gates["NAND"])
h2 = perceptron(X, *gates["OR"])
H = np.column_stack([h1, h2])
y_xor = perceptron(H, *gates["AND"])
```

Параметры логических элементов

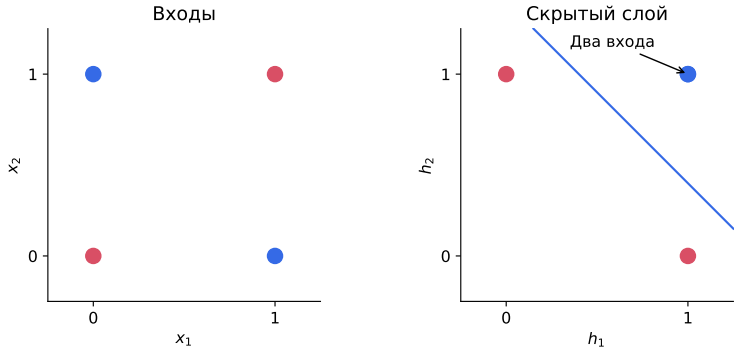
	w	b
AND	$(0.5, 0.5)$	-0.7
NAND	$(-0.5, -0.5)$	0.7
OR	$(0.5, 0.5)$	-0.4

`gates` хранит эти пары весов и смещений.

Выход XOR: `[0 1 1 0]`.

Здесь параметры заданы вручную.

XOR: преобразование признаков



Входы $(1, 0)$ и $(0, 1)$ переходят в одну точку $(h_1, h_2) = (1, 1)$.

В скрытом пространстве классы разделяются прямой $h_1 + h_2 = 1.4$.

Подбор параметров по данным

Заданные параметры

Для простых логических функций мы сами выбирали параметры: **w**, b.

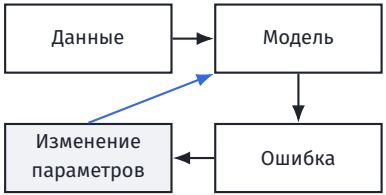
Но для реальной задачи:

- входных данных может быть очень много;
- параметров модели могут быть тысячи или миллионы;
- подходящие значения заранее неизвестны.

Обучаемые параметры

Параметры модели можно не задавать вручную.

Компьютер **автоматически изменяет w** и b, используя данные и информацию об ошибке.



2

Нейронные сети

Объединение нейронов в сеть

Один персептрон

Один персептрон вычисляет

$$z = \mathbf{w}^T \mathbf{x} + b$$

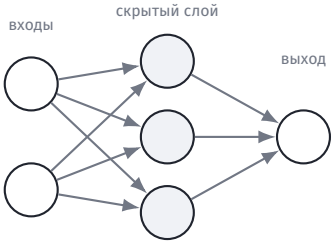
и принимает решение на основе значения z.

- один персептрон задаёт одну линейную границу решения;
- несколько персептронов можно объединять;
- параметры $\mathbf{w}$ и b можно обучать по данным.

Слои сети

Объединим нейроны в **слои**:

вход $\longrightarrow$ скрытые слои $\longrightarrow$ выход.



Нейрон как вычислительный блок

Нейрон выполняет два последовательных шага.

1. Линейная комбинация

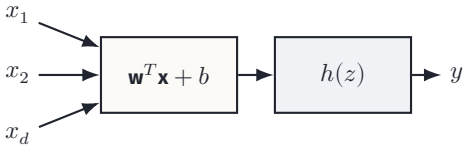
$$z = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b$$

или короче

$$z = \mathbf{w}^T \mathbf{x} + b.$$

2. Функция активации

$$y = h(z)$$



Функция активации

Для классического персептрона мы использовали **ступенчатую функцию**:

$$h(z) = \begin{cases} 0, & z \leq 0, \\ 1, & z > 0. \end{cases}$$

Она сразу превращает число z в решение:

$$z \quad \longrightarrow \quad h(z) \quad \longrightarrow \quad 0 \text{ или } 1.$$

Без активации

Нейрон выполнял бы только

$$z = \mathbf{w}^T \mathbf{x} + b,$$

то есть линейное преобразование.

С активацией

Функция $h(z)$ позволяет добавить **нелинейность** и строить более сложные зависимости.

Линейные и нелинейные слои

Представим два последовательных слоя без функций активации:

$$\mathbf{z}_1 = W_1\mathbf{x} + \mathbf{b}_1,$$

$$\mathbf{z}_2 = W_2\mathbf{z}_1 + \mathbf{b}_2.$$

Подставим первое выражение во второе:

$$\mathbf{z}_2 = W_2\left(W_1\mathbf{x} + \mathbf{b}_1\right) + \mathbf{b}_2 = W\mathbf{x} + \mathbf{b}$$

— снова одно линейное преобразование.

Нелинейная активация

Если между слоями использовать нелинейную функцию

$$\mathbf{h}_1 = h\left(W_1\mathbf{x} + \mathbf{b}_1\right),$$

то несколько слоёв уже нельзя свести к одному линейному преобразованию.

Примеры функций активации

Step

$$h(x) = \begin{cases} 0, & x \leq 0, \\ 1, & x > 0. \end{cases}$$

Резкое переключение между двумя состояниями.

Использовалась в классическом персептроне.

Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Плавно отображает вход в интервал

$$0 < \sigma(x) < 1.$$

Часто используется на выходе при бинарной классификации.

ReLU

$$\mathrm{ReLU}(x) = \max(0, x)$$

$$\mathrm{ReLU}(x) = \begin{cases} 0, & x \leq 0, \\ x, & x > 0. \end{cases}$$

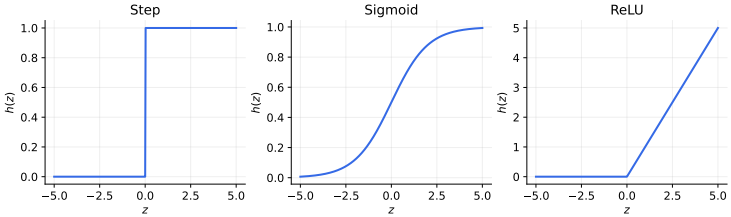
Широко используется в скрытых слоях нейронных сетей.

Step, Sigmoid и ReLU в NumPy

```python
def sigmoid(x):
    return np.exp(-np.logaddexp(0.0, -np.asarray(x)))

def step(x):
    return (np.asarray(x) > 0).astype(int)

def relu(x):
    return np.maximum(0, x)
```



В коде sigmoid вычисляется через `logaddexp`, чтобы избежать переполнения экспоненты.

Матричная запись слоя

Если в слое несколько нейронов, у каждого есть свои веса и смещение.

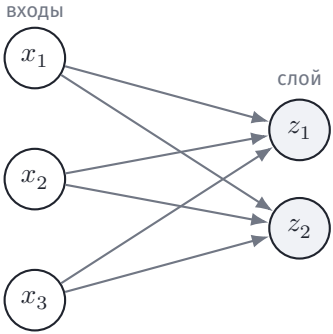
$$z_1 = \mathbf{w}_1^T \mathbf{x} + b_1,$$
$$z_2 = \mathbf{w}_2^T \mathbf{x} + b_2.$$

Обе формулы удобно объединить:

$$\mathbf{z} = W\mathbf{x} + \mathbf{b}$$

где

$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{bmatrix}.$$



Матричное умножение в NumPy

Пусть

$$M = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \qquad N = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}.$$

```
M = np.array([[1, 2], [3, 4]])
N = np.array([[5, 6], [7, 8]])
print("M @ N:\n", M @ N)
print("M * N:\n", M * N)
```

Тогда

$$MN = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}.$$

В NumPy оператор

$$M@N$$

означает матричное умножение.

Число столбцов первой матрицы должно совпадать с числом строк второй.

Один объект и пакет объектов

В формулах: объект-столбец

$$z = W_{\text{col}} x + b.$$

В NumPy: объекты по строкам

$$Z = XW + b, \qquad W = W_{\text{col}}^{T}.$$

$X\colon B \times d, W\colon d \times n, \quad b\colon 1 \times n.$

Смещение прибавляется к каждой строке.

$$Z : B \times n, \qquad A = h(Z).$$

```
X = np.array([[1, 2]])
W = np.array([[1, 3, 5], [2, 4, 6]])
print("X, W, X @ W:", X.shape, W.shape, (X @ W).
    shape)
print(X @ W)
```

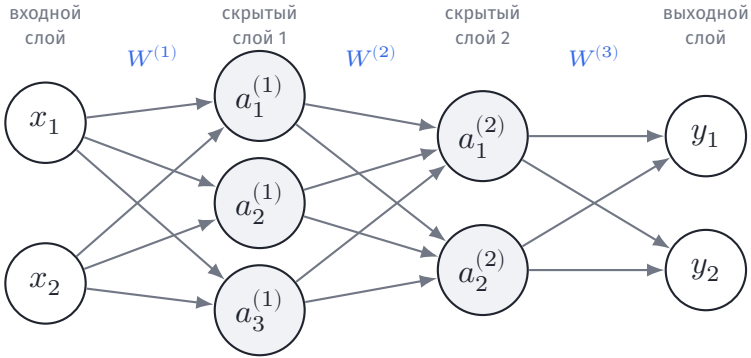
Результат: `[[5 11 17]]`

Z — до активации, A — после неё.

3

Многослойные нейронные сети

Структура многослойной нейронной сети



Кружки обозначают выходы нейронов $a^{(l)}$, а переход между слоями задаётся матрицей весов $W^{(l)}$.

Прямой проход: сеть 2 → 3 → 2 → 1

Последовательность слоёв

$$
\begin{aligned}
Z_1 &= XW_1 + b_1, \quad & A_1 &= \sigma(Z_1),\\
Z_2 &= A_1W_2 + b_2, \quad & A_2 &= \sigma(Z_2),\\
Z_3 &= A_2W_3 + b_3, \quad & Y &= \sigma(Z_3).
\end{aligned}
$$

Размеры весов: $2 \times 3, 3 \times 2, 2 \times 1$.

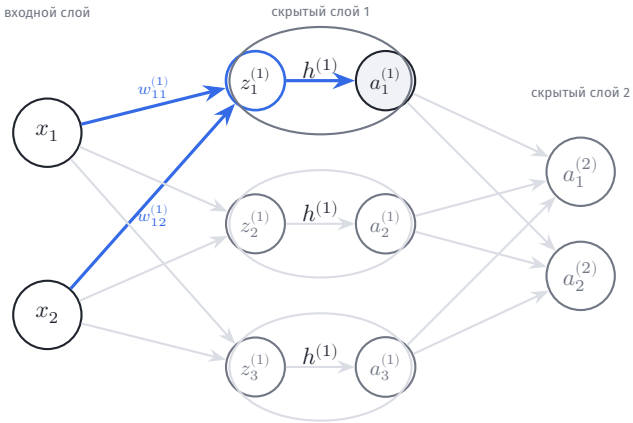
$X = (0.5 \quad 1.0)$.

Параметры заданы вручную; обучение пока не выполняется.

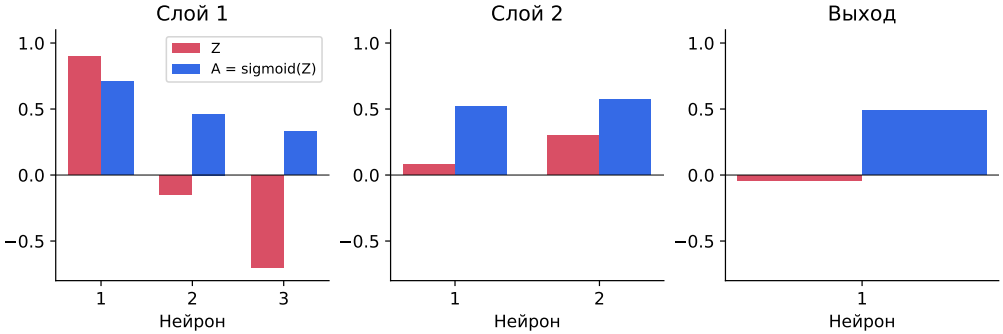
```
Z1 = X @ W1 + b1
A1 = sigmoid(Z1)
Z2 = A1 @ W2 + b2
A2 = sigmoid(Z2)
Z3 = A2 @ W3 + b3
Y = sigmoid(Z3)
```

Выход сети: $Y \approx 0.48983$.

Вычисления внутри скрытого слоя



Значения до и после активации



Первый слой: $Z_1 = (0.9,\ -0.15,\ -0.7), \quad A_1 \approx (0.711,\ 0.463,\ 0.332).$

Sigmoid применяется отдельно к каждому элементу.

Функция активации выходного слоя

Выбор функции активации в последнем слое зависит от типа задачи.

Регрессия

Прогнозируется вещественное число.

Обычно используется тождественная функция:

$$\boxed{\hat{y} = z}$$

Пример:

$$T = 325.4 \text{ К}.$$

Бинарная классификация

Два класса:

$$0 \quad \text{или} \quad 1.$$

Используется sigmoid:

$$\boxed{\hat{y} = \frac{1}{1 + e^{-z}}}$$

$$0 < \hat{y} < 1.$$

Многоклассовая классификация

Один из нескольких классов.

Используется softmax:

$$\boxed{\hat{y}_k = \frac{e^{z_k}}{\sum_i e^{z_i}}}$$

причём

$$\sum_k \hat{y}_k = 1.$$

4

Обучение нейронных сетей

Обучение нейронной сети

До сих пор мы вычисляли выход сети при уже известных параметрах:

$$\mathbf{x} \longrightarrow \text{нейронная сеть} \longrightarrow \hat{\mathbf{y}}$$

Но значения матриц $W^{(l)}$ и векторов $\mathbf{b}^{(l)}$ заранее неизвестны.

Цель обучения

Найти такие параметры сети, при которых предсказание $\hat{\mathbf{y}}$ как можно лучше соответствует правильному ответу $\mathbf{t}$.

$$(\mathbf{x}, \mathbf{t}) \to \hat{\mathbf{y}} \to L(\hat{\mathbf{y}}, \mathbf{t}) \to \text{изменение } W, \mathbf{b} \to \text{повторение}$$

- сеть делает предсказание;
- функция потерь измеряет ошибку;
- вычисляются градиенты;
- параметры изменяются так, чтобы уменьшить ошибку.

Функция потерь в наших примерах

Регрессия и учебный XOR

$$L = \frac{1}{2B} \sum_{i=1}^{B} (\hat{y}_i - y_i)^2.$$

Коэффициент $1/2$ сокращается при дифференцировании.

$$\frac{\partial L}{\partial \hat{y}_i} = \frac{\hat{y}_i - y_i}{B}.$$

Один выход на объект, B — размер batch.

Бинарная классификация Часто используют бинарную кросс-энтропию:

$$L_{\text{BCE}} = -\frac{1}{B} \sum_i \big[y_i \ln p_i + (1 - y_i) \ln(1 - p_i) \big].$$

В примере XOR оставляем $\frac{1}{2}$MSE, чтобы явно проследить производную sigmoid.

Число loss сравнивают только при одинаковом определении функции.

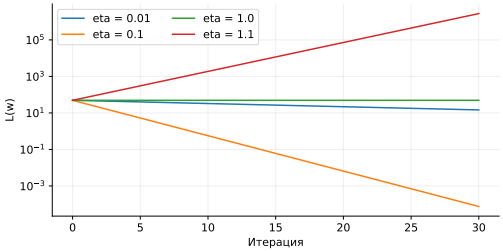
Скорость обучения на квадратичной функции

$$L(w) = (w-3)^2, \qquad w_0 = -4.$$

$$w \leftarrow w - \eta\, 2(w-3).$$

- $\eta = 0.01$: медленное сближение;
- $\eta = 0.1$: более быстрый спад;
- $\eta = 1$: колебания без уменьшения loss;
- $\eta = 1.1$: расходимость.

Эти границы относятся к данной функции.



Численный градиент

```
def loss_vec(w):
    return w[0]**2 + 3 * w[1]**2

def numeric_gradient(function, w, delta=1e-5):

    result = np.zeros_like(w, dtype=float)
    for i in range(w.size):
        plus, minus = w.copy(), w.copy()
        plus[i] += delta
        minus[i] -= delta
        result[i] = (function(plus) -
     function(minus)) / (2 * delta)
    return result
```

$$L(w_1, w_2) = w_1^2 + 3w_2^2.$$

В точке $(1, -2)$:

$$\nabla L = (2, -12).$$

Центральная разность:

$$\frac{L(w + \delta e_i) - L(w - \delta e_i)}{2\delta}.$$

При $\delta = 10^{-5}$ абсолютная ошибка меньше 10^{-8}.

Численный градиент удобен для проверки реализации.

Обратное распространение градиента

Сначала сеть вычисляет предсказание и значение функции потерь:

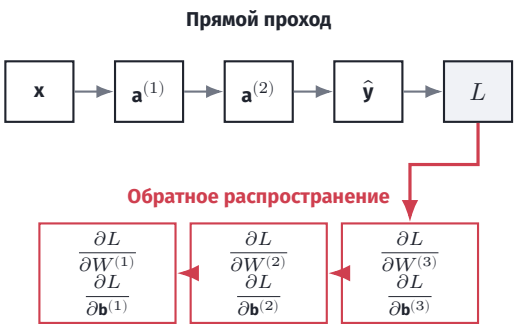
$$\mathbf{x} \to \hat{\mathbf{y}} \to L.$$

Но значение L только показывает, **насколько велика ошибка**.

Чтобы изменить параметры сети, необходимо узнать:

$$\frac{\partial L}{\partial W^{(l)}}, \qquad \frac{\partial L}{\partial \mathbf{b}^{(l)}}.$$

Эти градиенты вычисляются **от выходного слоя к входному** с помощью правила цепочки.



Backpropagation на одном нейроне

Рассмотрим один нейрон:

$$z = wx + b, \qquad a = h(z), \qquad L = L(a).$$

Чтобы определить влияние веса w на L, используем **правило цепочки**:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a}\frac{\partial a}{\partial z}\frac{\partial z}{\partial w}, \quad \frac{\partial L}{\partial b} = \frac{\partial L}{\partial a}\frac{\partial a}{\partial z}\frac{\partial z}{\partial b}$$

Так как

$$\frac{\partial a}{\partial z} = h'(z), \qquad \frac{\partial z}{\partial w} = x,$$

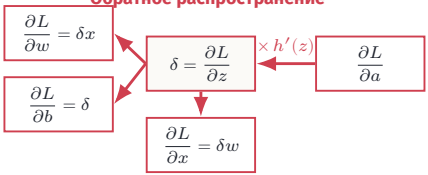
получаем

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a}\, h'(z)\, x, \quad \frac{\partial L}{\partial b} = \frac{\partial L}{\partial a}\, h'(z)$$

Прямой проход



Обратное распространение



Проверка backpropagation

Сеть XOR до обучения

- вычисляем аналитические градиенты;
- по очереди возмущаем каждый параметр;
- сравниваем с центральной разностью.

Проверяются все 13 параметров сети $2 \to 3 \to 1$.

$\delta = 10^{-5}$.

Максимальная абсолютная ошибка

Параметры	Ошибка
W_1	$1.44 \cdot 10^{-12}$
b_1	$2.44 \cdot 10^{-12}$
W_2	$2.08 \cdot 10^{-12}$
b_2	$6.79 \cdot 10^{-13}$

Градиенты вычисляются до обновления весов.

Train, validation и test

Train	Обновление весов и смещений.
Validation	Выбор скорости обучения, архитектуры или момента остановки.
Test	Итоговая оценка после фиксации всех решений.

В линейной регрессии: 96 объектов train, 32 validation, 32 test.

Если train loss уменьшается, а validation loss растёт, возможно переобучение.

В XOR обучаем все четыре комбинации. Его таблица не служит независимым тестом.

Линейная регрессия: обновление параметров

Синтетические данные

$$y = 2x - 1 + \varepsilon, \qquad \varepsilon \sim N(0, 0.3^2).$$

Модель: $\hat{y} = wx + b$.

80 эпох, batch из 16 объектов.

Скорость выбираем по validation из $0.001,\ 0.01,\ 0.1$.

После каждой эпохи loss пересчитывается на всей выборке.

```
error = w * xb + b - yb
dw = np.mean(error * xb)
db = np.mean(error)
w -= lr * dw
b -= lr * db
```

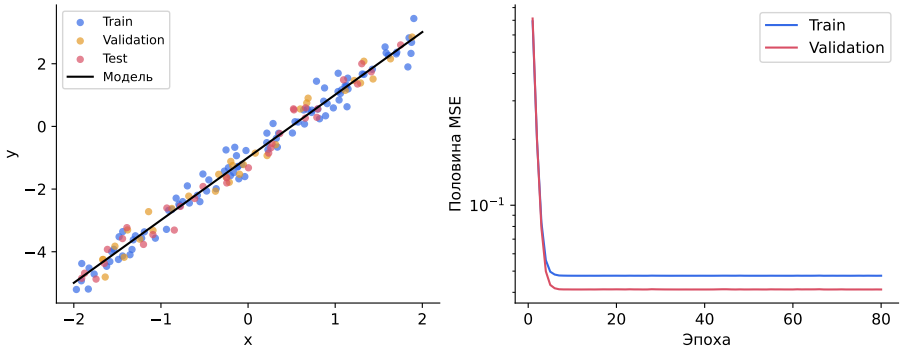
Выбрано $\eta = 0.1$.

$$w \approx 2.00113, \qquad b \approx -0.99007.$$

Минимум квадратов на том же train:

$$w \approx 2.00733, \qquad b \approx -0.98943.$$

Регрессия: результат и ошибка



Test RMSE: 0.28768. Тестовая выборка не использовалась для выбора скорости обучения.

5

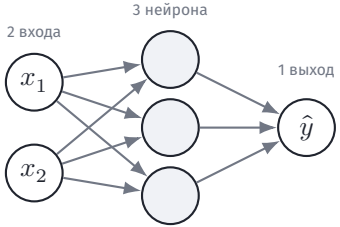
Практика и итоги

Сеть для XOR

Задача

Обучим нейронную сеть воспроизводить функцию XOR:

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0



Все четыре комбинации используются для обучения.

Случайная инициализация: `default_rng(0)`. Два слоя sigmoid.

XOR: прямой проход

```python
def forward(X, parameters):
    W1, b1, W2, b2 = parameters
    Z1 = X @ W1 + b1
    A1 = sigmoid(Z1)
    Z2 = A1 @ W2 + b2
    prediction = sigmoid(Z2)
    return A1, prediction
```

Размеры массивов

X	$B \times 2$
W_1, b_1	$2 \times 3, 1 \times 3$
Z_1, A_1	$B \times 3$
W_2, b_2	$3 \times 1, 1 \times 1$
$\hat{y}$	$B \times 1$

@ суммирует вклады входов; активация действует поэлементно.

XOR: градиенты в NumPy

```python
def loss_gradients(X, y, parameters):
    W1, b1, W2, b2 = parameters
    A1, prediction = forward(X, parameters)
    loss = 0.5 * np.mean((prediction - y)**2)
    dZ2 = (prediction - y) * prediction * (1 -
     prediction) / y.size
    dW2 = A1.T @ dZ2
    db2 = dZ2.sum(axis=0, keepdims=True)
    dZ1 = (dZ2 @ W2.T) * A1 * (1 - A1)
    dW1 = X.T @ dZ1
    db1 = dZ1.sum(axis=0, keepdims=True)
    return loss, (dW1, db1, dW2, db2)
```

Правило цепочки

$$\sigma'(z) = a(1-a), \quad a = \sigma(z).$$

`dZ2` — ошибка на выходном слое.

`dZ2 @ W2.T` передаёт её скрытому слою.

`sum(axis=0)` складывает вклады объектов в градиент смещения.

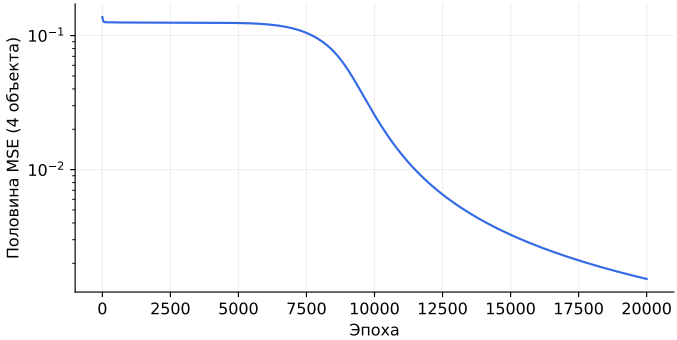
`y.size` учитывает усреднение в loss.

XOR: цикл обучения

```
lr = 0.1
epochs = 20000
batch_size = 2
loss_history = [loss_gradients(X_all, y_all, parameters)[0]]
for epoch in range(epochs):
    order = rng.permutation(len(X_all))
    for start in range(0, len(X_all), batch_size):
        ids = order[start:start + batch_size]
        _, gradients = loss_gradients(X_all[ids], y_all[ids], parameters)
        for parameter, gradient in zip(parameters, gradients):
            parameter -= lr * gradient
    loss_history.append(loss_gradients(X_all, y_all, parameters)[0])
```

Loss в истории относится ко всем четырём точкам после обновления параметров.

XOR: уменьшение функции потерь



20 000 эпох, $\eta = 0.1$, batch из двух объектов. $L : 0.1377 \longrightarrow 0.001528.$

XOR: проверка результата

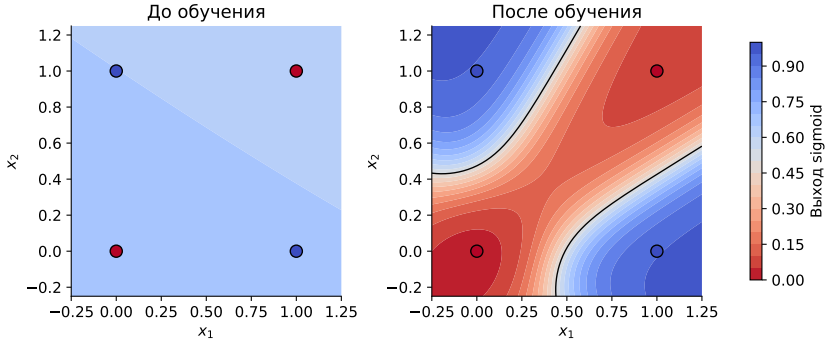
```
_, prediction = forward(X_all, parameters)
classes = (prediction >= 0.5).astype(int)
```

(x_1, x_2)	y	$\hat{y}$	Класс
$(0, 0)$	0	0.03020	0
$(0, 1)$	1	0.94304	1
$(1, 0)$	1	0.94548	1
$(1, 1)$	0	0.07140	0

Все четыре комбинации воспроизведены.

Это проверка обучающей таблицы XOR, а не оценка обобщения.

XOR: граница решения



Чёрная линия: выход sigmoid равен 0.5. Метки заданы только в четырёх точках; между ними показано продолжение модели.

Итоги лекции

- Персептрон задаёт линейную границу; скрытый слой позволяет реализовать XOR.
- Прямой проход вычисляет прогноз, обратный проход — градиенты.
- Скорость обучения влияет на сходимость; loss считаем по всей выборке.
- Train, validation и test решают разные задачи.

github.com/lyushisyan/computational-methods-course